Parallel Methods for Mining Top-K Probabilistic Positive–Negative High-Utility Itemsets from Uncertain Databases: A Shared-Memory Design-Space Study

Huy Vo[1], Nguyen Le[1†], Thien Nguyen[2*†]

[1]Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam.

[2]Natural Language Processing and Knowledge Discovery Laboratory, Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam.

*Corresponding author(s). E-mail(s): nguyenchithien@tdtu.edu.vn;

†These authors contributed equally to this work.

Abstract

Top-**K** high-utility itemset mining frees the analyst from choosing a minimum utility threshold; mining over *uncertain* databases models the existence probabilities that arise from noisy or aggregated data; and admitting *negative* item profits captures losses that co-occur with gains. Their combination defines the Top-**K** Probabilistic Positive–Negative High-Utility Itemset (PPNHUI) problem. Because negative profits invalidate the utility-based anti-monotonicity that ordinarily prunes the search, the candidate space is substantially enlarged, so efficient parallel execution becomes essential—yet its effectiveness is far from self-evident: the top-**K** admission threshold rises only as high-utility itemsets are discovered, at a rate that is strongly data-dependent. We define a common mining core built on a utility–probability list, the UPU-List, equipped with a three-tier pruning scheme, and prove that all three bounds remain valid upper bounds when item profits may be negative. Against an exact sequential baseline, we instantiate this core under four parallelisation strategies that span two design axes, namely threshold sharing and load balancing. A work-ratio metric, defined as the number of list-join operations performed relative to the sequential baseline, separates algorithmic pruning from raw hardware parallelism. Experiments on four benchmark datasets reveal a clear regime spectrum. On dense data, cooperative threshold escalation lets the work-stealing strategy perform as little as one

tenth of the sequential work and attain up to **27.5**× speedup on eight threads, exceeding linear scaling; on sparse data the same strategy performs more work than the baseline, and a producer–consumer strategy proves the most robust. The non-sharing strategy is consistently the weakest and can run slower than the sequential baseline. The work-ratio metric accounts for the entire spectrum and indicates which parallel strategy is preferable for a given data regime.

Keywords: high-utility itemset mining, uncertain databases, top-**K** mining, negative utility, parallel algorithms

1 Introduction

High-utility itemset mining (HUIM) generalises frequent itemset mining by weighting items with utilities such as profit, revealing patterns that are valuable rather than merely frequent [1, 2]. Three extensions move HUIM closer to real applications. First, the analyst rarely knows a good minimum-utility threshold; *top*-K mining instead asks directly for the K itemsets of highest utility [3, 4]. Second, data gathered from sensors, imputation, or aggregation is *uncertain*: each item in a transaction carries an existence probability [5]. Third, real profit tables contain *negative* entries—loss leaders, penalties, adverse events—so a faithful model must allow items with negative utility [6, 7].

Combining all three yields the Top-K Probabilistic Positive–Negative High-Utility Itemset (PPNHUI) problem. Negative items prevent the search from being pruned by utility alone and so enlarge it, which motivates parallel execution. However, whether parallelism actually helps is subtle. Top-K algorithms prune using an admission threshold that equals the K-th best utility found *so far*; this threshold starts low and rises only as good itemsets are discovered [4]. In a parallel execution many branches are explored before the threshold has risen, creating redundant work; conversely, if several workers *share* the threshold, an early discovery by one worker can prune the others. Which effect dominates is data-dependent, and negative items, by depressing utilities, slow the threshold's ascent. A study of parallel PPNHUI mining therefore needs a metric that separates algorithmic pruning from raw hardware speedup.

This paper provides such a study for the *shared-memory* setting, where the exact, instantaneous sharing of a global threshold is possible—something a distributed engine cannot easily offer. Our contributions are:

1. We formalise the Top-K PPNHUI problem over uncertain databases (Section 3) and a common mining core built on a utility–probability list (UPU-List) with a three-tier (EP, $PTWU$, PUB) pruning scheme (Section 4).
2. We prove that all three pruning tiers remain valid upper bounds when item profits may be negative and that the resulting search is exact (Lemmas 1–3, Theorem 4).
3. Against an exact sequential baseline, we instantiate the core under *four* shared-memory parallel strategies that span two design axes—threshold sharing and load balancing—and analyse them with a *work-ratio* metric.

4. On four datasets we show a regime spectrum: cooperative threshold escalation yields up to 27.5× (super-linear) speedup on dense data but degrades on sparse data, where a producer–consumer strategy is most robust; the non-sharing strategy is always worst and can be slower than sequential (Sections 6–7).

2 Related Work

Top-*K *HUIM.

Top-K HUIM removes the minimum-utility threshold. Two-phase methods such as TKU [3] and REPT [8] first generate candidates and then verify them, while one-phase methods such as TKO [4] and KHMC [9] use a vertical utility-list [1] and raise the threshold during a single traversal. All of these target *certain* databases with non-negative utilities. Top-K mining remains active: recent advances include a metaheuristic top-K miner (TKU-BChOA [19]), threshold-raising with the pre-large concept on incremental data [22], incremental top-K mining over an accumulated database [21], top-K cross-level mining over data streams [20], and join-free utility-list traversal [24].

HUIM over uncertain databases.

Lin et al. [5] introduced potential high-utility itemset mining (PHUIM) for uncertain data, with the level-wise PHUI-UP and the list-based PHUI-List built on a probability-utility (PU) list; MUHUI [10] adds pruning to the same structure. These methods mine *all* potential high-utility itemsets rather than a top-K set, and assume non-negative profits. Most recently, Nguyen and Nguyen [18] mine top-K high-utility itemsets from uncertain databases using dynamic minimum-probability and utility thresholds—the closest prior setting to ours, yet still without negative profits or any parallel treatment.

HUIM with negative utilities.

HUINIV-Mine [6] first handled negative item values; FHN [7] introduced a positive-and-negative utility list (PNU-list) that is far faster, and surveys document the area [11]. Most recently, TopHUI [12] performs *top*-K HUIM with negative utility—but for *certain* databases; other recent methods mine negatives from unstable negative-profit databases [23] and high-utility sequences with positive and negative values [25]. Our UPU-List combines the PU-list idea of [5] with the positive/negative bookkeeping of [7]; the structure itself is therefore evolutionary rather than novel, which is why our contribution is the parallel study, not the data structure.

Parallel and distributed HUIM.

Big-data HUIM has largely been distributed: PHUI-Growth on Hadoop [13], PKU for parallel top-K HUIM on Spark [14], P-FHM+ on Spark [15], GPU-based heuristics [16], Hadoop-based P-EFIM [26], and more recent Spark-based parallel HUIM [27]. These frameworks partition the work into largely independent subtasks and therefore cannot share an exact global threshold across nodes. Shared-memory multi-core HUIM has received far less attention, and to our knowledge no prior work studies how the choice

of shared-memory parallelisation strategy interacts with top-K threshold dynamics under *uncertain, positive–negative* utilities. That gap is the subject of this paper.

3 Preliminaries and Problem Statement

3.1 Basic Concepts

Let $I = \{i_1, \ldots, i_m\}$ be a set of items. A *profit table* assigns each $i \in I$ a value $pr(i) \in \mathbb{R}$; unlike classical HUIM, $pr(i)$ may be negative.

Definition 1 (Uncertain transaction database) $\mathcal{D} = \{T_1, \ldots, T_n\}$ where each transaction T_d associates with every item $i \in T_d$ a quantity $q(i, T_d) \in \mathbb{Z}^+$ and an existence probability $p(i, T_d) \in (0, 1]$. Item probabilities are assumed independent.

Definition 2 (Utility) $u(i, T) = pr(i) \cdot q(i, T)$ and, for $X \subseteq T$, $u(X, T) = \sum_{i \in X} pr(i)\, q(i, T)$, which may be negative.

Definition 3 (Probabilities) $\mathrm{P}(X, T) = \prod_{i \in X} p(i, T)$ for $X \subseteq T$ (and 0 otherwise). The *expected utility* and *existential probability* of X are

$$EU(X) = \sum_{T \supseteq X} u(X, T)\, \mathrm{P}(X, T), \qquad EP(X) = 1 - \prod_{T \supseteq X} \big(1 - \mathrm{P}(X, T)\big). \tag{1}$$

Because $u(X, T)$ may be negative, $EU(X)$ may be negative.

Problem 1 (Top-K PPNHUI mining) Given $\mathcal{D}$, a profit table pr (possibly negative), $\mathrm{K} \geq 1$ and $minProb \in [0, 1]$, let $\mathcal{V} = \{X \subseteq I : X \neq \emptyset,\ EP(X) \geq minProb\}$. Return $\mathcal{R} \subseteq \mathcal{V}$ with $|\mathcal{R}| = \min(\mathrm{K}, |\mathcal{V}|)$ such that $EU(X) \geq EU(Y)$ for every $X \in \mathcal{R}$ and $Y \in \mathcal{V} \setminus \mathcal{R}$.

Remark 1 (Negative-utility itemsets) Problem 1 places no sign constraint on EU: when fewer than K valid itemsets have non-negative expected utility, the least-negative ones are returned. This matches the positive–negative setting and our implementation. If an application requires $EU \geq 0$, one adds that constraint to $\mathcal{V}$ and lower-bounds the admission threshold at 0.

3.2 Illustrative Example

Table 1 shows a small uncertain database over items $\{a, b, c\}$ with profits $pr(a) = 5$, $pr(b) = -3$, $pr(c) = 2$ (so b is a negative-profit item); each entry lists an item's quantity and existence probability. Applying Definition 3, Table 2 gives EU and EP for every itemset. With $minProb = 0.7$ all six itemsets are valid, and ranking them by EU gives $\{a\} > \{a, c\} > \{c\} > \{a, b\} > \{b, c\} > \{b\}$. For $\mathrm{K} \leq 4$ the answer contains only positive-EU itemsets, but for $\mathrm{K} = 5$ it must include $\{b, c\}$ ($EU = -2.75$) and for $\mathrm{K} = 6$ also $\{b\}$ ($EU = -9.3$): a concrete instance of Remark 1. Note also that $EP(\{b\}) = 1$ because b occurs with probability 1 in transaction T_4.

Table 1 Example uncertain database over items $\{a, b, c\}$ with profits $pr(a) = 5$, $pr(b) = -3$, $pr(c) = 2$. Each cell lists (quantity, probability); a dash means the item is absent. The last column gives the positive transaction utility $PTU(T) = \sum_{i \in T,\, pr(i) > 0} pr(i)\, q(i, T)$, which seeds the $PTWU$ bound.

TID	a	b	c	$PTU(T)$
T_1	$(1, 0.9)$	$(2, 0.8)$	$(1, 1.0)$	7
T_2	$(2, 0.6)$	–	$(3, 0.7)$	16
T_3	–	$(1, 0.5)$	$(2, 0.9)$	4
T_4	$(1, 0.8)$	$(1, 1.0)$	–	5
Total ($= PTWU(\varnothing)$)				32

Table 2 Expected utility, existential probability and the $PTWU$ upper bound for every itemset of the database in Table 1, in descending EU order. Column $|X|$ is the itemset size. Note $EU \leq PTWU$ in every row (Lemma 2). Negative-EU itemsets are shown in italics; with K = 4 the top-K set is the first four rows, whereas K = 5 and K = 6 must admit the negative-EU itemsets $\{b, c\}$ and $\{b\}$.

Itemset	$\lvert X\rvert$	EP	EU	$PTWU$
$\{a\}$	1	0.992	14.50	28
$\{a, c\}$	2	0.942	13.02	23
$\{c\}$	1	1.000	9.80	27
$\{a, b\}$	2	0.944	0.88	12
$\{b, c\}$	2	0.890	−2.75	11
$\{b\}$	1	1.000	−9.30	16

4 Proposed Methods

4.1 Mining Core and Upper Bounds

The sequential baseline and the four parallel strategies share one mining core and differ only in how Phase 3 is parallelised (Figure 1). Phase 1 scans $\mathcal{D}$ once to compute per-item bounds and probabilities, discards items that cannot belong to any valid pattern, and builds a UPU-List per surviving item. Phase 2 seeds the result with the valid 1-itemsets. Phase 3 is a depth-first traversal of the set-enumeration tree that extends each prefix by joining UPU-Lists and applies three pruning tiers.

For an itemset X, the UPU-List stores, per transaction $T \supseteq X$, the utility $u(X, T)$, a positive remaining utility $rem(X, T)$, and $\log \mathrm{P}(X, T)$ (log-space for stability); two

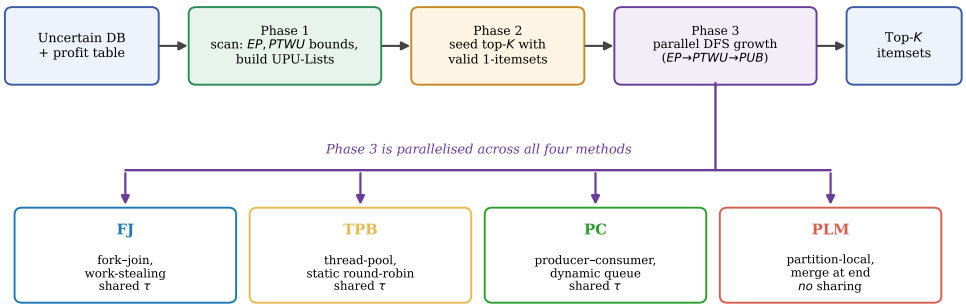


Fig. 1 Overview of the mining pipeline. The three-phase core (build, seed, prune-and-grow) is shared by all methods; only Phase 3—the depth-first pattern-growth traversal—is parallelised, and the four strategies differ in how they share the admission threshold τ and balance prefix tasks.

lists are joined by a linear two-pointer merge (Figure 2). Concretely, to extend a prefix X by an item e (giving $Z = X \cup \{e\}$), the merge keeps only transactions t present in both lists and, on each such matching t, combines the entries by

$$\begin{aligned} u(Z,t) &= u(X,t) + u(e,t), \qquad & rem(Z,t) &= \min\big(rem(X,t), rem(e,t)\big), \\ \log \mathrm{P}(Z,t) &= \log \mathrm{P}(X,t) + \log \mathrm{P}(e,t). \end{aligned} \tag{2}$$

During the same pass the four cached aggregates are accumulated as

$$EU(Z) = \sum_t u(Z,t)\, e^{\log \mathrm{P}(Z,t)}, \qquad PTWU(Z) = \min\big(PTWU(X), PTWU(e)\big), \tag{3}$$

$$PUB(Z) = \sum_{t:\ u(Z,t)+rem(Z,t)>0} e^{\log \mathrm{P}(Z,t)} \big(u(Z,t) + rem(Z,t)\big), \tag{4}$$

$$EP(Z) = 1 - \prod_t \big(1 - e^{\log \mathrm{P}(Z,t)}\big). \tag{5}$$

where $EP(Z)$ is accumulated in log-space for numerical stability. These are exactly the operations annotated in Figure 2. Fix a total order $\prec$ on items (descending $PTWU$). With $PTU(T) = \sum_{i \in T, pr(i)>0} pr(i) q(i,T)$,

$$PTWU(X) = \sum_{T \supseteq X} PTU(T), \qquad rem(X,T) = \sum_{i \in T,\ i \succ X,\ pr(i)>0} pr(i) q(i,T), \tag{6}$$

$$PUB(X) = \sum_{T \supseteq X:\ u(X,T)+rem(X,T)>0} \mathrm{P}(X,T)\big(u(X,T) + rem(X,T)\big). \tag{7}$$

An *extension* of X is any $Z \supseteq X$ whose extra items follow X under $\prec$.

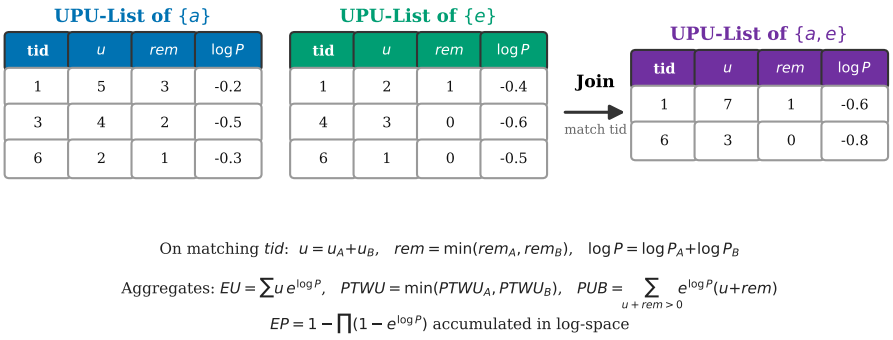


Fig. 2 The UPU-List structure and the two-pointer Join. Entries sharing a transaction id are merged and the cached aggregates EU, EP, $PTWU$ and PUB are updated; the merge and aggregation rules are given in Eqs. (2)–(5).

Lemma 1 (EP anti-monotonicity) *If $X \subseteq Z$ then $EP(Z) \leq EP(X)$.*

Proof $\mathrm{P}(Z,T) \leq \mathrm{P}(X,T)$ as each $p(i,T) \in (0,1]$, so $1-\mathrm{P}(Z,T) \geq 1-\mathrm{P}(X,T)$ for all T and the product over T is no smaller for Z; hence $EP(Z) \leq EP(X)$. □

Lemma 2 ($PTWU$ bound) *$EU(Z) \leq PTWU(Z) \leq PTWU(X)$ for any $X \subseteq Z$.*

Proof For $T \supseteq Z$, $u(Z,T)\mathrm{P}(Z,T) \leq \max\{u(Z,T),0\} \leq PTU(T)$: if $u(Z,T) \geq 0$ then multiplying by $\mathrm{P}(Z,T) \in (0,1]$ does not increase it, otherwise the left side is negative; and the positive part of $u(Z,T)$ is at most the positive utilities of T. Summing over $T \supseteq Z$ gives $EU(Z) \leq PTWU(Z)$. Since $\{T \supseteq Z\} \subseteq \{T \supseteq X\}$ and $PTU \geq 0$, $PTWU(Z) \leq PTWU(X)$. □

Lemma 3 (PUB bound) *For every extension Z of X, $EU(Z) \leq PUB(X)$.*

Proof For $T \supseteq Z$ the extra items of Z lie in the $\prec$-suffix, so their positive contribution is at most $rem(X,T)$ and $u(Z,T) \leq u(X,T) + rem(X,T)$; also $\mathrm{P}(Z,T) \leq \mathrm{P}(X,T)$ by Lemma 1. Hence $u(Z,T)\mathrm{P}(Z,T) \leq \max\{0, u(X,T)+rem(X,T)\}\,\mathrm{P}(X,T)$. Summing over $T \supseteq Z$ and dropping the non-positive terms gives $EU(Z) \leq PUB(X)$. □

Theorem 4 (Correctness) *Pruning X when $EP(X) < minProb$, $PTWU(X) < \tau$, or $PUB(X) < \tau$—where τ is the current* K*-th largest admitted EU—never discards an itemset of an exact top-*K *solution. Hence the search is exact.*

Algorithm 1 GROW: depth-first pattern growth from a prefix

Require: prefix P, start index s, ordered items σ, 1-itemset lists $\mathcal{L}$, collector C, *minProb*

$\tau \leftarrow C$.THRESHOLD()
if $P.PTWU < \tau$ **then return**
end if
for $j \leftarrow s$ **to** $|\sigma| - 1$ **do**
  $J \leftarrow$ JOIN(P, $\mathcal{L}[\sigma[j]]$, τ)
  if $J =$ NULL **or** $J.EP < minProb$ **or** $J.PTWU < \tau$ **or** $J.PUB < \tau$ **then continue**
  end if
  if $J.EU \geq \tau$ **then** C.TRYCOLLECT(J)
  end if
  GROW(J, j+1, σ, $\mathcal{L}$, C, *minProb*)
end for

Proof τ only increases and never exceeds the final K-th best EU. By Lemma 1 an itemset failing the EP test and all its extensions are invalid. By Lemmas 2–3, $PTWU(X) < \tau$ or $PUB(X) < \tau$ implies $EU(Z) \leq \tau$ for every extension Z, so none can displace a current top-K member. Thus only subtrees lying entirely below the final threshold are pruned. □

Remark 2 (Why sharing helps and why negatives matter) Theorem 4 holds for any τ not exceeding the final K-th best EU. Raising τ earlier—as when many workers feed a shared collector—prunes more without ever discarding a solution; a thread-local threshold is simply a smaller, also-safe τ that prunes less. Negative items depress EU and slow the ascent of τ, which both reduces the benefit of sharing and increases redundant parallel exploration.

4.2 Pattern Growth

Algorithm 1 is the shared growth procedure; τ is read from a top-K collector holding the current K highest-EU itemsets.

The growth procedure relies on two further operations: the list JOIN (Algorithm 2), which merges a prefix with an extension item in one linear pass while computing EU, EP, $PTWU$ and PUB; and TRYCOLLECT (Algorithm 3), the thread-safe admission of a candidate into the shared top-K heap. TRYCOLLECT uses a volatile threshold for a lock-free fast reject and takes the lock only when a candidate might actually be admitted; this is what lets the shared-threshold strategies propagate τ cheaply across workers.

4.3 Parallelisation Strategies

The *baseline* SEQ is a single-thread depth-first pattern-growth miner over all 1-itemset prefixes: it follows the utility-list pattern-growth paradigm of HUI-Miner [1] with the single-pass top-K threshold-raising of TKO [4], extended to negative profits in the manner of TopHUI [12] and to uncertain data following PHUIM [5, 18]. SEQ is

Algorithm 2 JOIN: merge a prefix list with an extension-item list

Require: prefix list A, extension list B (item e), threshold τ

$\rho \leftarrow \min(A.PTWU,\ B.PTWU)$
if $\rho < \tau$ **then return** NULL
end if ▷ Tier 2, before the merge
$eu \leftarrow 0$; $ub \leftarrow 0$; $\ell c \leftarrow 0$; $L \leftarrow$ empty list ▷ $\ell c = \sum \log(1 - \mathrm{P})$
$i \leftarrow 0$; $j \leftarrow 0$
while $i < |A|$ **and** $j < |B|$ **do**
  if $A.tid_i = B.tid_j$ **then**
    $u \leftarrow A.u_i + B.u_j$; $r \leftarrow \min(A.r_i, B.r_j)$; $\ell p \leftarrow A.\ell p_i + B.\ell p_j$
    append $(A.tid_i,\ u,\ r,\ \ell p)$ to L
    $eu \leftarrow eu + u \cdot e^{\ell p}$
    if $u + r > 0$ **then** $ub \leftarrow ub + e^{\ell p}(u + r)$
    end if
    $\ell c \leftarrow \ell c + \log(1 - e^{\ell p})$
    $i \leftarrow i + 1$; $j \leftarrow j + 1$
  else if $A.tid_i < B.tid_j$ **then** $i \leftarrow i + 1$
  else $j \leftarrow j + 1$
  end if
end while
if L is empty **then return** NULL
end if
return list for $A \cup \{e\}$ with $EU{=}eu$, $EP{=}1 - e^{\ell c}$, $PTWU{=}\rho$, $PUB{=}ub$

Algorithm 3 TRYCOLLECT: admit a candidate into the shared top-K

Require: candidate J; min-heap H ordered by EU; size K; volatile threshold τ

if $J.EU < \tau$ **then return**
end if ▷ lock-free fast reject
acquire lock
if $J.EU \geq \tau$ **then**
  H.PUSH(J)
  if $|H| >$ K **then** H.POPMIN()
  end if
  $\tau \leftarrow (|H| < \mathrm{K})\ ?\ -\infty\ :\ H.\mathrm{MIN}().EU$
end if
release lock

the exact sequential reference and the work/time baseline against which the parallel strategies are measured.

Building on this core, the *four* parallel strategies span two axes—whether the admission threshold is *shared*, and how prefix tasks are *balanced* (Figure 3):

- **FJ** — fork–join: prefixes are recursively split into balanced batches on a work-stealing pool; one shared collector, so any worker's discovery raises τ for all.

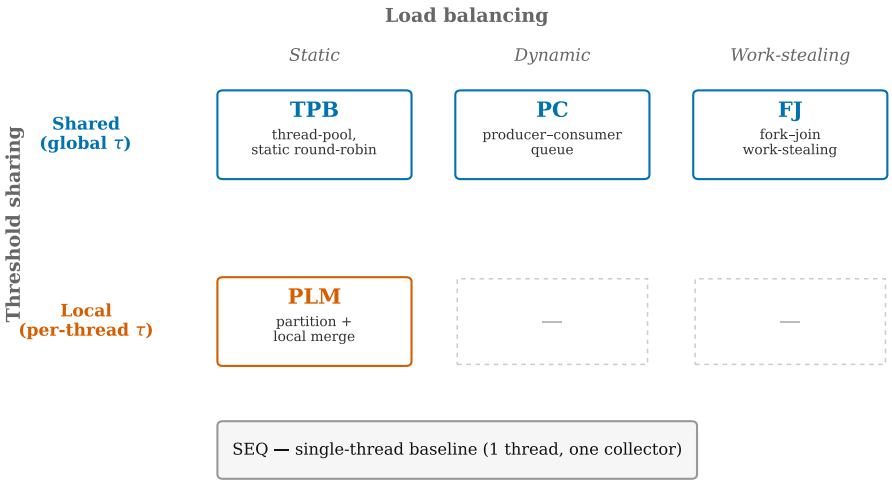


Fig. 3 The shared-memory design space. The four parallel strategies are placed by whether they share the global admission threshold τ (rows) and how they balance prefix tasks (columns). SEQ is the single-thread baseline, shown for reference.

- **TPB** — thread-pool: prefixes assigned round-robin by $PTWU$ rank to a fixed pool; shared collector, static one-prefix tasks.
- **PLM** — partition–local merge: $PTWU$-balanced blocks, each mined into a *thread-local* collector, merged at the end; no threshold sharing.
- **PC** — producer–consumer: a producer enqueues prefix tasks on a bounded queue, consumers mine into a shared collector; dynamic scheduling.

By Remark 2, FJ, TPB and PC prune against the global K-th best, whereas PLM prunes against weaker local thresholds. SEQ runs the prefixes of Phase 3 in a single loop, and TPB submits one static GROW task per prefix (in $PTWU$ order) to a fixed pool that shares the collector; both are straightforward and omitted. The three non-trivial schedulers are given in Algorithms 4–6. FJ (Algorithm 4) recursively halves the prefix range on a work-stealing pool, balancing load dynamically while sharing one collector. PLM (Algorithm 5) gives each thread a $PTWU$-balanced block and its *own* collector, merging them only at the end. PC (Algorithm 6) feeds prefixes through a bounded queue to consumer threads that share the collector.

5 Experimental Setup

All algorithms are implemented in Java and share an identical mining core, so performance differences isolate the parallelisation strategy.

Datasets and profit/probability assignment.

We use four standard transactional benchmarks obtained from the SPMF open-source data-mining repository [17] (Table 3): Chess and Mushroom, both originally from the

Algorithm 4 ForkJoinMine (FJ): recursive prefix-batch scheduling

Require: prefix range $[lo, hi)$, ordered items σ, lists $\mathcal{L}$, *shared* collector C, *minProb*

if $hi - lo \leq 1$ **then**
  for $p \leftarrow lo$ **to** $hi - 1$ **do**
    Grow($\mathcal{L}[\sigma[p]]$, $p{+}1$, σ, $\mathcal{L}$, C, *minProb*)
  end for
  return
end if
$m \leftarrow \lfloor (lo + hi)/2 \rfloor$
fork ForkJoinMine($[lo, m), \dots$) ▷ stolen by an idle worker
ForkJoinMine($[m, hi), \dots$) ▷ run on the current worker
join the forked task

Algorithm 5 PartitionLocalMerge (PLM): no threshold sharing

Require: ordered items σ, lists $\mathcal{L}$, size K, *minProb*, threads B

partition $\{0, \dots, |\sigma| - 1\}$ into $PTWU$-balanced blocks $\Pi_1, \dots, \Pi_B$
create thread-local collectors $C_1, \dots, C_B$, each of size K
for $b \leftarrow 1$ **to** B **in parallel do**
  for $p \in \Pi_b$ **do**
    Grow($\mathcal{L}[\sigma[p]]$, $p{+}1$, σ, $\mathcal{L}$, C_b, *minProb*)
  end for
end for
$C \leftarrow$ empty top-K; **for** $b = 1$ **to** B: C.merge(C_b)
return C

UCI Machine Learning Repository; the Retail dataset of a Belgian retail store; and Liquor. They span three orders of magnitude in density. Chess and Mushroom are dense, fixed-length transactions; Retail and Liquor are sparse. To obtain a positive–negative profit table, each item is assigned a profit and a fraction of items is made negative, giving the negative fractions (%neg, 30–51%) in Table 3; profit ranges and average positive/negative profits are summarised in Table 4. For Chess, Mushroom and Retail the profits follow the standard SPMF utility generation and the 30–51% of items with the lowest utility are negated; for Liquor each profit is the item's real average utility (so positive profits can be large, up to 14,615) and the 30% lowest-utility items are negated.

Regarding uncertainty, all four datasets carry genuine per-item existence probabilities taken from their SPMF uncertain versions; the probabilities are intrinsic to the data and are not assigned by us. In every dataset a substantial fraction of items have an existence probability of exactly 1.0. Because expected utility scales with existence probability (Definition 3), itemsets formed from such probability-one items attain the highest expected utility and are therefore admitted into the top-K ahead of itemsets that contain lower-probability items. On Chess, Mushroom and Retail the top-K result is consequently *dominated* by high-probability itemsets, so the existence-probability

Algorithm 6 ProducerConsumerMine (PC): dynamic queue scheduling

Require: ordered items σ, lists $\mathcal{L}$, *shared* collector C, *minProb*, threads N

$Q \leftarrow$ bounded blocking queue
producer: enqueue $0, \ldots, |\sigma| - 1$ into Q, then N poison tokens
for each of N consumers **in parallel do**
  loop
    $p \leftarrow Q$.take()
    if p is a poison token **then**
      break
    end if
    Grow($\mathcal{L}[\sigma[p]]$, $p{+}1$, σ, $\mathcal{L}$, C, *minProb*)
  end loop
end for

Table 3 Characteristics of the experimental datasets. $|T|$: transactions; $|I|$: distinct items; avg$|t|$, max$|t|$: average and maximum transaction length; Density = avg$|t|/|I|$; $|I^-|$: negative-profit items; %neg: their fraction.

Dataset	$\lvert T\rvert$	$\lvert I\rvert$	avg$\lvert t\rvert$	max$\lvert t\rvert$	Density	$\lvert I^-\rvert$	%neg	Source
Chess	3,196	75	37.00	37	49.33%	38	50.7%	SPMF
Mushroom	8,124	119	23.00	23	19.33%	45	37.8%	SPMF
Retail	88,162	16,470	10.31	76	0.063%	8,247	50.1%	SPMF
Liquor	52,131	4,026	7.87	11	0.196%	1,207	30.0%	SPMF

Table 4 Profit-table statistics and the observed EU range of the top-K results (over the swept K).

Dataset	Profit range	Avg. pos.	Avg. neg.	EU range of top-K
Chess	[−63.0, +48.0]	+16.22	−17.05	[43,091, 151,130]
Mushroom	[−60.0, +90.0]	+18.41	−17.38	[61,761, 451,145]
Retail	[−130.0, +130.0]	+17.26	−17.17	[3,348, 1,229,567]
Liquor	[−27.5, +14,615.0]	+148.61	−17.55	[661, 560,291]

filter (*minProb* = 0.7) is seldom the *binding* constraint for the winning itemsets—though it remains active throughout and prunes lower-probability candidates. On Liquor the high-utility items span a wider probability range, so the probabilistic ranking and the *EP* filter shape the result more visibly. This is a property of which itemsets win the top-K competition under the expected-utility ranking, not a degeneracy of the data: every dataset used here is genuinely uncertain.

Parameters.

Because the appropriate K differs by density, we sweep K $\in \{10, 20, 30, 40, 50\}$ for Chess, $\{30, \ldots, 70\}$ for Mushroom, $\{500, \ldots, 900\}$ for Retail, and $\{600, \ldots, 1000\}$ for

Liquor; $minProb = 0.7$ throughout. SEQ is the single-thread baseline and the parallel strategies are run at $\{2, 4, 8\}$ threads.

Metrics.

We report wall-clock time, speedup relative to SEQ, and a *work ratio*: the number of UPU-List joins performed by a strategy divided by that of SEQ at the same (K, *minProb*). The work ratio separates algorithmic effects—how much of the search tree is explored, which threshold sharing changes—from raw hardware parallelism. Speedup is reported against the single-thread SEQ baseline so that a value above the thread count indicates super-linear behaviour.

Hardware and software.

All experiments were run on a laptop with a 12th-generation Intel Core i7-12700H processor: a hybrid design with 6 performance cores (each two-way simultaneously multithreaded) and 8 efficiency cores, for 14 physical cores and 20 hardware threads in total, at a 2.70 GHz base frequency. The machine has 16 GB of RAM (15.7 GB usable) and runs 64-bit Windows 11. The implementation targets the Java Virtual Machine (OpenJDK 17); no GPU was used. Because we evaluate $\{2, 4, 8\}$ software threads, the work-stealing and thread-pool schedulers map onto the performance cores first and then the efficiency cores; the hybrid topology and laptop thermal envelope are noted as threats to validity (Section 7).

Protocol.

Each (algorithm, K, *minProb*, threads) configuration was executed three times after a warm-up run to allow just-in-time compilation and heap stabilisation, and we report the *median* of the three timed runs. Run-to-run variation was low: the median coefficient of variation across all configurations was 1.8% (mean 3.4%). The work-stealing strategy FJ, whose stealing order is nondeterministic, exhibited the largest variance (coefficient of variation up to $\approx$26% on Liquor and $\approx$12% on Chess at eight threads); reporting medians rather than means controls for these outliers. All strategies are exact: at every configuration they returned the same top-K set as SEQ, so timing differences reflect only the parallelisation strategy and not result quality.

6 Results

Across all datasets and configurations every strategy returned exactly K patterns identical to SEQ, consistent with Theorem 4. We focus on performance.

Table 5 reports speedup and work ratio at eight threads for the largest K of each dataset. Figure 4 shows speedup versus K (all K values) at eight threads, and Figure 5 shows the work ratio versus K.

Dense data (Chess, Mushroom).

FJ's work ratio falls far below one (to 0.11 on Chess at K = 50), i.e. cooperative threshold escalation prunes roughly nine tenths of the sequential work. Combined with eight-way hardware parallelism this yields *super-linear* speedup, up to 27.5× on

Table 5 Speedup (s) and work ratio (w) at 8 threads, largest K per dataset. Work ratio < 1 means a strategy performs less work than SEQ.

Dataset	K	SEQ (ms)	FJ (s/w)	TPB (s/w)	PC (s/w)	PLM (s/w)
Chess	50	69509	**27.5**/0.11	4.7/1.19	4.7/1.23	0.7/7.92
Mushroom	70	7841	**10.0**/0.44	8.4/0.52	8.6/0.54	0.9/4.28
Liquor	1000	8499	2.4/1.19	3.1/1.09	**3.9**/1.07	2.4/1.66
Retail	900	87442	2.2/1.18	2.7/1.11	**3.4**/1.12	0.6/8.95

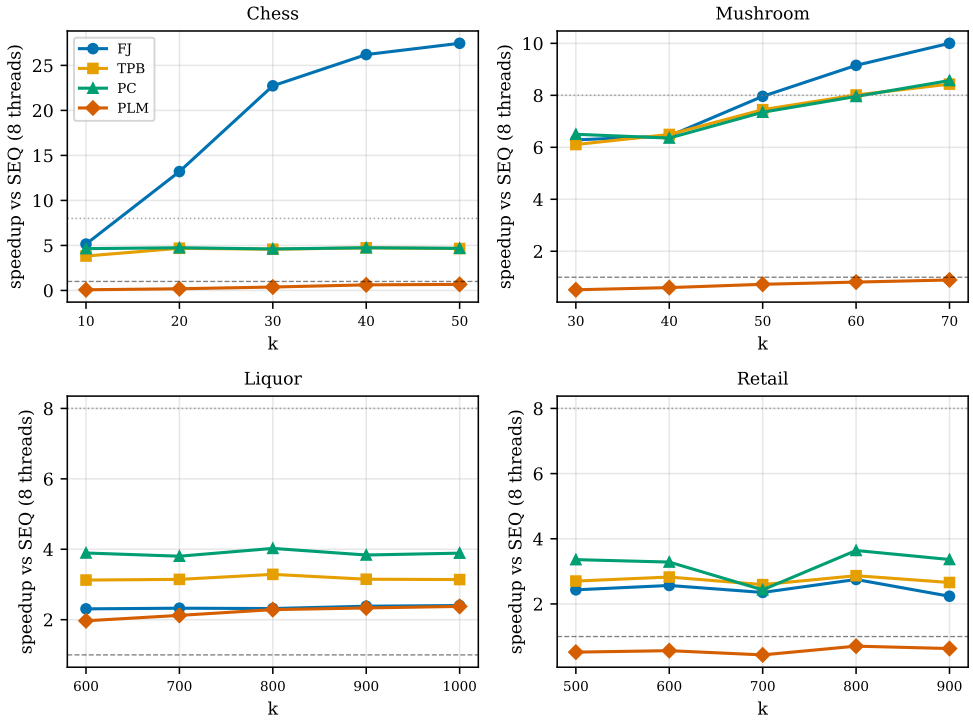


Fig. 4 Speedup over SEQ versus K at eight threads, for all K values per dataset. The dotted line marks eight-way (linear) speedup, the dashed line marks parity with SEQ. FJ is super-linear on the dense datasets and its advantage grows with K, but it is sub-linear on the sparse datasets, where PC scales best.

eight threads. TPB and PC also scale well ($\approx 4.7\times$ on Chess, $\approx 8.5\times$ on Mushroom) but with higher work ratios than FJ (≈ 1.2 on Chess, ≈ 0.5 on Mushroom), so they capture less of the cooperative-pruning windfall.

Sparse data (Liquor, Retail).

Here FJ's work ratio exceeds one (1.13–1.32 over the sweep): the threshold rises slowly, so parallel branches explore redundantly before pruning takes effect, and FJ's speedup

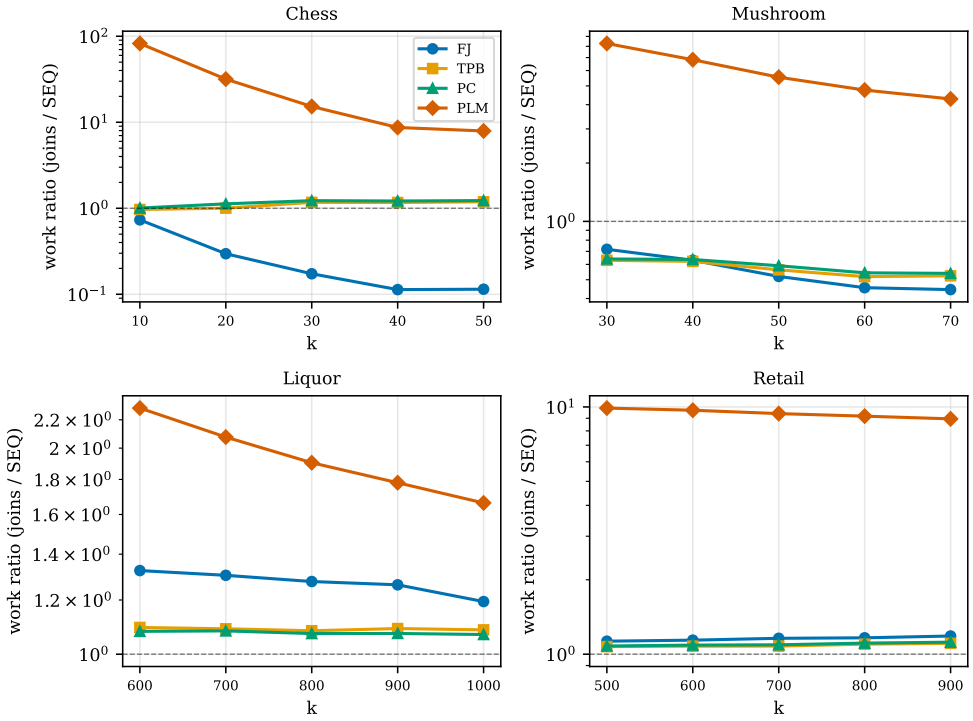


Fig. 5 Work ratio (joins relative to SEQ, log scale) versus K at eight threads. Values below the dashed line (= 1) mean less work than sequential. FJ falls well below 1 on dense data and rises above 1 on sparse data; PLM is always highest.

collapses to ≈ 2.2–2.8×. The producer–consumer strategy PC is the most robust, reaching ≈ 3.4–4.0× with a work ratio near one.

The non-sharing strategy.

PLM is the worst strategy in every regime, and on Chess, Mushroom and Retail it runs *slower than sequential*. Its work ratio climbs steeply as K shrinks: on Chess it reaches ≈ 80× the sequential work at K = 10 (Figure 5), so PLM can be more than 10× slower than SEQ (speedup as low as 0.07×); even at the largest K it still does 4–9× the work (speedup 0.6–0.9×). The penalty tracks how much pruning the shared threshold enables—largest where sharing matters most, and smallest on Liquor, where PLM stays near a 2× speedup.

7 Discussion

The work-ratio metric explains the whole spectrum. The benefit of sharing the admission threshold is proportional to how quickly that threshold rises. On dense data a few high-utility itemsets are found early, τ rises sharply, and a strategy that propagates

τ aggressively (FJ with work-stealing) converts this into a large reduction in explored nodes—enough to make speedup super-linear. On sparse data the utility landscape is flatter, τ rises slowly, sharing buys little, and FJ's eager parallel splitting instead creates redundant work; the more measured scheduling of PC then wins. PLM, which never shares, forfeits whatever pruning the global threshold would have provided, so it is worst everywhere and worst of all where that pruning is largest.

Two design lessons follow. First, no single strategy dominates: practitioners should pick the strategy from the data regime, and the work ratio—measurable on a small sample—predicts which regime applies. Second, super-linear speedup in top-K mining is an *algorithmic* effect (cooperative pruning), not a hardware one; reporting wall-clock speedup alone is misleading, and the work ratio is needed to attribute it correctly.

On attribution: density, not the negative fraction. The regime tracks structural density almost perfectly (Table 3): the two datasets where FJ wins, Chess and Mushroom, have densities of 49.3% and 19.3%, whereas the two where PC wins, Retail and Liquor, are three orders of magnitude sparser (0.063% and 0.196%). The proportion of negative-profit items does *not* explain the split: Chess (50.7% negative) and Retail (50.1% negative) carry almost the same negative fraction yet fall in opposite regimes, while Liquor has the fewest negatives (30.0%) and still behaves like Retail. Thus, although negative items slow the ascent of τ all else equal (Remark 2), across these datasets it is density—through its effect on how fast a few high-utility itemsets push τ up—that decides which parallel strategy wins, not the negative fraction itself. Disentangling any residual effect of the negative fraction would require holding density fixed while varying it, which we leave to future work.

Threats to validity.

The item profits are assigned rather than intrinsic (except for Liquor, whose profits are real average utilities), so they may not reflect a specific application. The existence probabilities, by contrast, are intrinsic to the SPMF datasets; because the highest-utility itemsets on Chess, Mushroom and Retail happen to be composed of probability-one items, our results on these three datasets primarily exercise the density and positive–negative dimensions, while Liquor most strongly exercises the probabilistic ranking. The hardware is a single hybrid-core laptop CPU under thermal constraints (see Section 5), which can add run-to-run variance, particularly for the nondeterministic work-stealing strategy; we mitigate this by reporting medians over repeated runs. Finally, PLM's partitioning on one dataset (Retail) showed an unusually high work ratio that merits closer inspection.

8 Conclusion

We have studied the design space of shared-memory parallel mining for the Top-K Probabilistic Positive–Negative High-Utility Itemset problem, in which top-K selection, uncertain data and negative profits are combined for the first time under a unified parallel treatment. Our starting point is a common mining core built on the UPU-List, a utility–probability list that fuses the probabilistic and the positive/negative bookkeeping of prior structures. We proved that its three pruning bounds—the

existential-probability test, the positive transaction-weighted utility, and the positive upper bound—remain valid upper bounds even when item profits may be negative, so that the resulting top-K search is exact (Theorem 4).

On this core we instantiated four parallelisation strategies that span two design axes, threshold sharing and load balancing, and evaluated them against an exact sequential baseline. To attribute the observed speedups correctly we introduced a *work-ratio* metric, the number of list joins performed relative to the sequential baseline, which isolates algorithmic pruning from raw hardware parallelism. The central empirical finding is a clear *regime spectrum*. On dense data, sharing the admission threshold lets a few early discoveries raise the threshold sharply; a work-stealing scheduler propagates this aggressively and prunes up to nine tenths of the sequential work, turning eight-thread execution into *super-linear* speedup (up to 27.5×). On sparse data the threshold rises slowly, eager parallel splitting instead creates redundant work, and a more measured producer–consumer scheduler becomes the most robust choice. The strategy that never shares the threshold is uniformly the weakest and can run slower than the sequential baseline, with its penalty largest precisely where sharing would have helped most. A separate analysis shows that the regime is governed by structural *density* rather than by the negative-profit fraction: datasets with nearly identical negative fractions but very different densities fall into opposite regimes.

Two practical lessons follow. First, no single parallel strategy dominates; the appropriate strategy should be selected from the data regime, and the work ratio—measurable on a small sample—predicts which regime applies. Second, super-linear speedup in top-K mining is an algorithmic effect of cooperative pruning rather than a hardware effect, so reporting wall-clock speedup alone is misleading and the work ratio is needed to attribute it correctly.

Several directions remain. An adaptive scheduler could choose a strategy at runtime from a cheaply measured work ratio, combining the dense-regime advantage of work-stealing with the sparse-regime robustness of producer–consumer. Controlled experiments that hold density fixed while varying the negative fraction would isolate any residual effect of negative profits. Evaluating the strategies on a server-class processor with a higher physical core count, and developing a distributed extension that approximates global threshold sharing across nodes, would test how far the regime spectrum and the cooperative-pruning effect carry beyond the shared-memory, single-machine setting studied here.

Declarations

Funding. The authors received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors for this study.

Competing interests. The authors declare no competing interests.

Ethics approval and consent to participate. Not applicable. This study does not involve human participants, human data, or animals.

Consent for publication. Not applicable.

Data availability. All four base datasets (Chess, Mushroom, Retail and Liquor) are publicly available from the SPMF open-source data-mining repository [17]. The positive–negative profit tables and the exact uncertain versions used in our experiments were derived from these public datasets as described in Section 5 and are available from the corresponding author on reasonable request.

Code availability. The Java implementation of the four parallel strategies, the sequential baseline, and the shared mining core is available from the corresponding author on reasonable request.

Author contributions. H. Vo and N. Le contributed equally: they designed and implemented the algorithms, ran the experiments, and drafted the manuscript. T. Nguyen supervised the work, contributed to the methodology, and revised the manuscript. All authors read and approved the final version.

References

[1] Liu M, Qu J. Mining high utility itemsets without candidate generation. In: Proc. 21st ACM Int. Conf. on Information and Knowledge Management (CIKM); 2012. p. 55–64.

[2] Gan W, Lin JCW, Fournier-Viger P, Chao HC, Tseng VS, Yu PS. A survey of utility-oriented pattern mining. IEEE Transactions on Knowledge and Data Engineering. 2021;33(4):1306–1327.

[3] Wu CW, Shie BE, Tseng VS, Yu PS. Mining top-K high utility itemsets. In: Proc. 18th ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD); 2012. p. 78–86.

[4] Tseng VS, Wu CW, Fournier-Viger P, Yu PS. Efficient Algorithms for Mining Top-K High Utility Itemsets. IEEE Transactions on Knowledge and Data Engineering. 2016;28(1):54–67.

[5] Lin JCW, Gan W, Fournier-Viger P, Hong TP, Tseng VS. Efficient algorithms for mining high-utility itemsets in uncertain databases. Knowledge-Based Systems. 2016;96:171–187.

[6] Chu CJ, Tseng VS, Liang T. An efficient algorithm for mining high utility itemsets with negative item values in large databases. Applied Mathematics and Computation. 2009;215(2):767–778.

[7] Lin JCW, Fournier-Viger P, Gan W. FHN: An efficient algorithm for mining high-utility itemsets with negative unit profits. Knowledge-Based Systems. 2016;111:283–298.

[8] Ryang H, Yun U. Top-k high utility pattern mining with effective threshold raising strategies. Knowledge-Based Systems. 2015;76:109–126.

[9] Duong QH, Liao B, Fournier-Viger P, Dam TL. An efficient algorithm for mining the top-k high utility itemsets, using novel threshold raising and pruning strategies. Knowledge-Based Systems. 2016;104:106–122.

[10] Lin JCW, Gan W, Fournier-Viger P, Hong TP, Tseng VS. Efficiently mining uncertain high-utility itemsets. Soft Computing. 2017;21(11):2801–2820.

[11] Singh K, Singh SS, Kumar A, Biswas B. High utility itemsets mining with negative utility value: A survey. Journal of Intelligent & Fuzzy Systems. 2018;35(6):6551–6562.

[12] Gan W, Wan S, Chen J, Chen CM, Qiu L. TopHUI: Top-k high-utility itemset mining with negative utility. In: Proc. IEEE Int. Conf. on Big Data (Big Data). IEEE; 2020. p. 5350–5359.

[13] Lin YC, Wu CW, Tseng VS. Mining high utility itemsets in big data. In: Advances in Knowledge Discovery and Data Mining (PAKDD). Springer; 2015. p. 649–661.

[14] Lin CH, Wu CW, Huang JT, Tseng VS. Parallel Mining of Top-k High Utility Itemsets in Spark In-Memory Computing Architecture. In: Advances in Knowledge Discovery and Data Mining (PAKDD). Springer; 2019. p. 234–246.

[15] Sethi KK, Ramesh D, Edla DR. P-FHM+: Parallel high utility itemset mining algorithm for big data processing. Procedia Computer Science. 2018;132:918–927.

[16] Fang W, Jiang H, Lu H, Sun J, Wu X, Lin JCW. GPU-based efficient parallel heuristic algorithm for high-utility itemset mining in large transaction datasets. IEEE Transactions on Knowledge and Data Engineering. 2024;36(2):652–667.

[17] Fournier-Viger P, Gomariz A, Gueniche T, Soltani A, Wu CW, Tseng VS. SPMF: a Java open-source pattern mining library. Journal of Machine Learning Research. 2014;15:3389–3393.

[18] Nguyen K, Nguyen T. Algorithms Based on Dynamic Minimum Probabilistic Support and Utility Thresholds for Mining Top-K High-Utility Itemsets From Uncertain Databases. IEEE Access. 2025;13:165222–165246.

[19] Mofid A, Daneshpour N, Torabi Z, Taghavi P. TKU-BChOA: an accurate meta-heuristic method to mine Top-k high utility itemsets. The Journal of Supercomputing. 2024;80(14):21284–21305.

[20] Han M, Liu S, Gao Z, Mu D, Li A. Mining Top-K constrained cross-level high-utility itemsets over data streams. Knowledge and Information Systems. 2024;66(5):2885–2924.

[21] Lee C, Kim H, Cho M, Kim H, Vo B, Lin JCW, et al. Incremental Top-k High Utility Pattern Mining and Analyzing Over the Entire Accumulated Dynamic

Database. IEEE Access. 2024;12:77605–77620.

[22] Tung NT, Nguyen LTT, Nguyen TDD, Huynh B. Efficient mining top-k high utility itemsets in incremental databases based on threshold raising strategies and pre-large concept. Knowledge-Based Systems. 2025;310:112971.

[23] Tung NT, Nguyen TDD, Nguyen LTT, Vo B. An efficient method for mining high-utility itemsets from unstable negative profit databases. Expert Systems with Applications. 2024;237:121489.

[24] Yan Y, Niu X, Zhang Z, Fournier-Viger P, Ye L, Min F. Efficient high utility itemset mining without the join operation. Information Sciences. 2024;681:121218.

[25] Zhang X, Lai F, Chen G, Gan W. Mining high-utility sequences with positive and negative values. Information Sciences. 2023;637:118945.

[26] Cheng Z, Shen W, Fang W, Lin JCW. A Parallel High-Utility Itemset Mining Algorithm Based on Hadoop. Complex System Modeling and Simulation. 2023;3(1):47–58.

[27] Li C, Zhang L, Sun A. Parallel High Utility Itemset Mining Algorithm on the Spark. In: Advances in Intelligent Information Hiding and Multimedia Signal Processing. Springer; 2024. p. 124–134.